



PROGRAMAÇÃO I

Aula 03

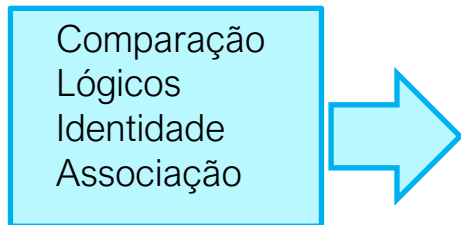
Vinicius Bischoff

Operadores são usados para realizar operações entre variáveis, constantes e expressões

O Python divide os operadores nos seguintes grupos:



Foram vistos na aula passada



Serão vistos na aula de hoje

Operadores deAtribuição

Operador	Nome
=	X= 1
+=	X= X+1
-=	X= X-1
*=	X= X*1
/=	X= X/1
%=	X= X%1

São utilizados no momento da **atribuição** de valores à variáveis e controlam como a atribuição será realizada.

Exemplo da utilização de cada operador de atribuição mencionado acima:

```
numero = 5  
numero += 7  
print(numero)  
# Resultado será 12
```

Operador +=:

```
numero = 5  
numero -= 3  
print numero  
# Resultado será 2
```

Operador -=:

```
numero = 5  
numero %= 2  
print numero #  
Resultado será 1
```

Operador %=:

Em Python, os operadores de **incremento** e **decremento** são escritos como `"+="` e `"-="`, respectivamente.

Ambos os operadores são usados para adicionar ou subtrair um valor de uma variável existente.

O operador `"+="` incrementa o valor atual da variável com um valor específico, enquanto o operador `"-="` diminui o valor atual da variável pelo mesmo valor.

- Suponha que você esteja construindo um programa de gerenciamento de estoque para uma loja. Você precisa manter um registro das unidades disponíveis de um produto específico.
- Inicialmente, a quantidade desse produto é 50. Quando um cliente compra uma unidade desse produto, você precisa atualizar o valor do estoque em seu sistema.
- Para isso, você pode usar o operador de decremento (`-=`) da seguinte maneira:

```
estoque = 50 # Estoque inicial do produto  
venda = 1 # Quantidade vendida  
estoque -= venda # Atualiza o estoque após a venda  
print("Novo estoque: ", estoque)
```

Nesse exemplo, o operador "`-=`" é usado para subtrair o número de unidades vendidas do valor atual do estoque. Depois, o novo valor do estoque é exibido na tela.

O operador de incremento (`+=`) funciona de forma semelhante, mas é usado para adicionar um valor a uma variável em vez de subtrair.

Operadores de Comparação

Operador	Nome	Sintaxe	Exemplo	Resultado
==	Igual	$x == y$	$10 == 3$	Falso
!=	Diferente	$x != y$	$12.6 != 5.1$	Verdadeiro
>	Maior que	$x > y$	$3 > 3.2$	Falso
<	Menor que	$x < y$	$4 < 4$	Falso
>=	Maior ou Igual	$x >= y$	'b' >= 'a'	Verdadeiro
<=	Menor ou Igual	$x <= y$	'A' <= 'a'	Verdadeiro

São usados para realizar comparação entre dois valores.

Vamos ver exemplos da utilização de cada operador de comparação mencionado acima.

Para facilitar o entendimento, todas as operações estão retornando um valor igual a True, para que você entenda como cada condição é aceita:

```
var = 5

if var == 5:
    print('Os valores são iguais')

if var != 7:
    print('O valor não é igual a 7')

if var > 2:
    print('O valor da variável é maior de 2')

if var >= 5:
    print('O valor da variável é maior ou igual a 5')

if var < 7:
    print('O valor da variável é menor que 7')

if var <= 5:
    print('O valor da variável é menor ou igual a 5')
```

```
Os valores são iguais
O valor não é igual a 5
O valor da variável é maior de 5
O valor da variável é maior ou igual a 5
O valor da variável é menor que 7
O valor da variável é menor ou igual a 5
```


Operadores Lógicos

Operador	Descrição	Sintaxe
and	Retorna verdadeiro se todos os elementos envolvidos forem verdadeiros	x and y
or	Retorna falso se todos os elementos envolvidos forem falsos	x or y
not	Retorna o inverso do elemento	not x

São usados para combinar expressões de comparação e lógicas.

Operadores Lógicos

Tabela Verdade: Operador AND

Condição A	Condição B	AND (A.B)
True	True	True
True	False	False
False	True	False
False	False	False

Tabela Verdade: Operador OR

Condição A	Condição B	OR (A + B)
True	True	True
True	False	True
False	True	True
False	False	False

Tabela Verdade: Operador NOT

Condição A	NOT (~A)
True	False
False	True

Tabela Verdade.

Exemplos

```
num1 = 7
num2 = 4

# Exemplo and
if num1 > 3 and num2 < 8:
    print("As Duas condições são verdadeiras")

# Exemplo or
if num1 > 4 or num2 ≤ 8:
    print("Uma ou duas das condições são verdadeiras")

# Exemplo not
if not (num1 < 30 and num2 < 8):
    print('Inverte o resultado da condição entre os parênteses')
```

Operadores de Identidade

São usados para verificar se dois objetos são iguais; não em conteúdo, mas se ocupam o mesmo local na memória

Operador	Descrição	Sintaxe
is	Retorna verdadeiro se as duas variáveis são o mesmo objeto	x is y
is not	Retorna verdadeiro se as duas variáveis não são o mesmo objeto	x is not y

Exemplos

```
lista = [1, 2, 3]
outra_lista = [1, 2, 3]
recebe_lista = lista

# Recebe True, pois são o mesmo objeto
print(f"São o mesmo objeto? {lista is recebe_lista}")

# Retorna False, pois são objetos diferentes
print(f"São o mesmo objeto? {lista is outra_lista}")
```

```
São o mesmo objeto? True
São o mesmo objeto? False
```

Operadores de Identidade

- Exemplo

```
x = ['a', 'b', 'c']  
y = ['a', 'b', 'c']  
z = x
```

```
print('x is z:', x is z)  
print('x is y:', x is y)  
print('x == y:', x == y)
```

```
print('x is not z:', x is not z)  
print('x is not y:', x is not y)  
print('x != y:', x != y)
```

Operadores de Associação

- São usados para verificar se um elemento pertence a um conjunto

Operador	Descrição	Sintaxe
in	Retorna verdadeiro se o elemento pertence ao conjunto	x in y
not in	Retorna verdadeiro se o elemento não pertence ao conjunto	x not in y

Operadores de Associação

Exemplo

```
x = ['a', 'b', 'c']

y = 'a'
print('{} in {}: {}'.format(y, x, y in x))
print('{} not in {}: {}'.format(y, x, y not in x))

y = 'd'
print('{} in {}: {}'.format(y, x, y in x))
print('{} not in {}: {}'.format(y, x, y not in x))
```

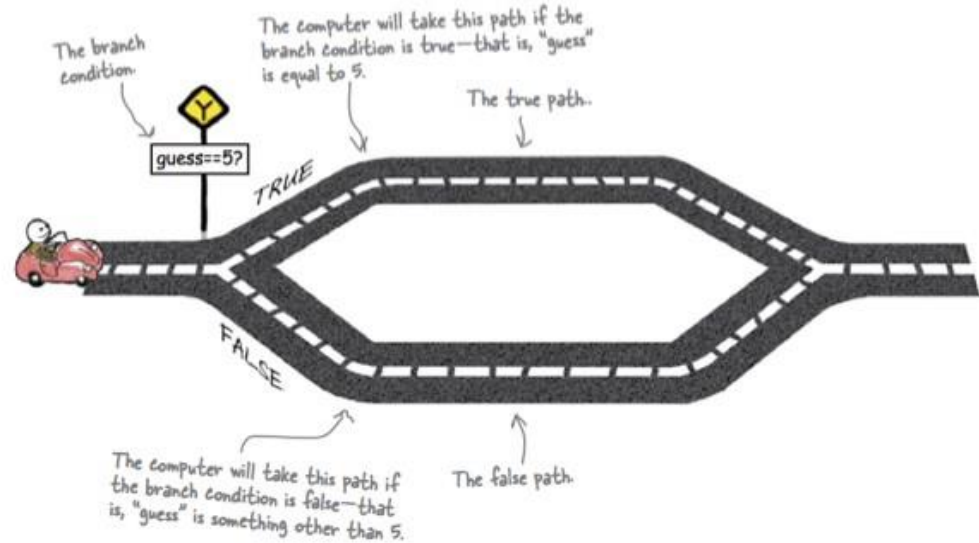

Precedência

- Os operadores obedecem a seguinte ordem de execução

Operador	Descrição
(), []	Parênteses, índice
**	Exponenciação
-X	Complemento unário – inversão do sinal
*, /, //, %	Multiplicação, divisão, divisão inteira e resto da divisão
+, -	Adição e subtração
in, not in, is, is not, <, <=, >, >=, !=, ==	Comparações incluindo identidade e associação
not	Lógica not
and	Lógica and
or	Lógica or
= %= /= //= -= += *= **=	Atribuição com e sem aritmética

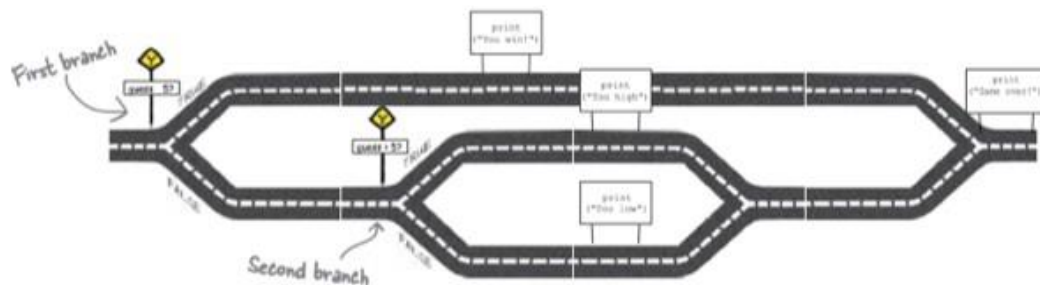
Estrutura de Seleção

A execução de um código
é semelhante a um
veículo transitando por
uma cidade



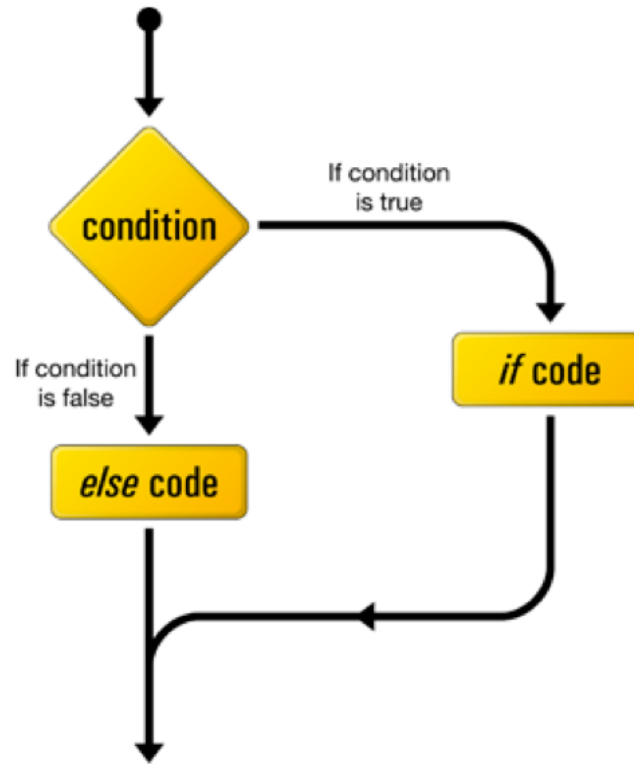
Estrutura de Seleção

Os caminhos podem conter mais de uma bifurcação, mas sempre se reencontram em um ponto comum



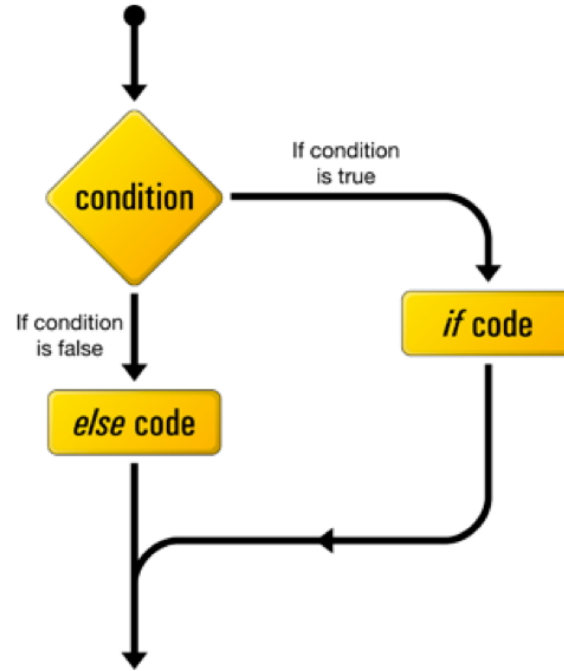
Comando IF

```
if condicao:  
    # uma ou mais instruções  
    # da condição verdadeira  
else:  
    # uma ou mais instruções  
    # da condição falsa
```



Comando IF Aninhado

```
if condicao_1:  
    # uma ou mais instruções  
    # da condição verdadeira  
elif condicao_2:  
    # uma ou mais instruções  
    # da condição falsa/verdadeira  
else:  
    # uma ou mais instruções  
    # da condição falsa/falsa
```



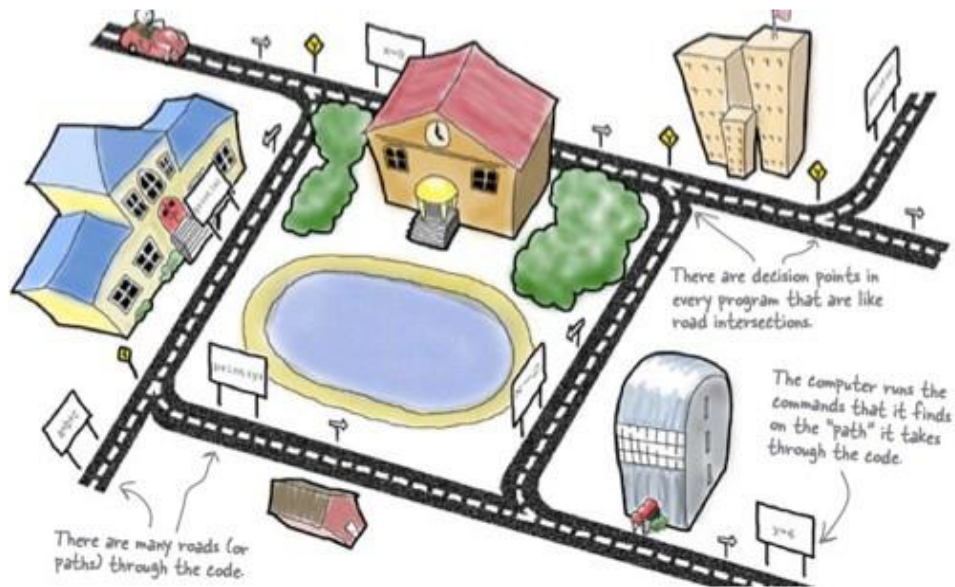
Exemplos do comando IF

```
x = input('Informe uma letra: ')
if x >= 'A' and x <= 'Z':
    print('Letra maiúscula')
else:
    print('Letra minúscula')
```

```
x = int(input('Informe um número: '))
if x < 5:
    print('Valor {} menor que 5'.format(x))
elif x > 5:
    print('Valor {} maior que 5'.format(x))
else:
    print('Valor {} igual a 5'.format(x))
```

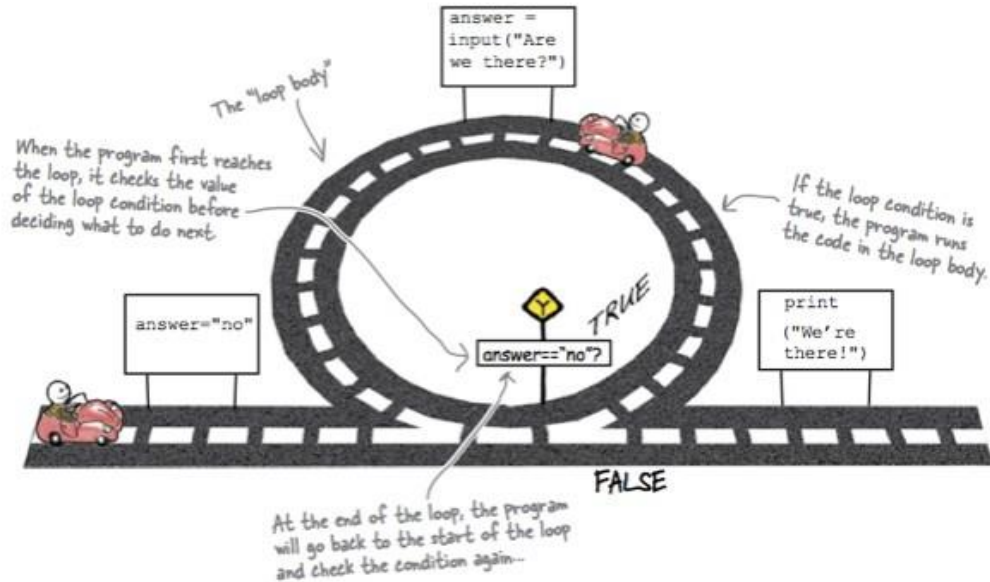
Estrutura de Repetição

A execução de um código é semelhante a um veículo transitando por uma cidade



Estrutura de Repetição

Em um determinado momento, chegamos em uma rotatória. Devemos entrar? Quantas vezes?



Estruturas de Repetição

- Alguns algoritmos podem ser muito repetitivos, por exemplo:
 - Geração de sequências numéricas
 - Números naturais, pares, ímpares, séries
 - Validações por tentativa e erro
 - Números primos, MMC
 - Processamento em lote
 - Entrada de dados em sequência, identificação de máximo e mínimo

Estruturas de Repetição

- Permitem que uma ou mais instruções sejam executadas
 - Uma **quantidade exata** de vezes

```
para nr de 1 até 10 passo 1:  
    mostre nr
```

- **Enquanto** uma determinada **condição** for satisfeita

```
resp = 'x'  
enquanto !(resp == 's' ou resp == 'n')  
    resp = leia('Deseja continuar? s/n')
```

Estruturas de Repetição

- **Contada**

- Executa uma **quantidade exata** de vezes
- Usa uma variável de controle do tipo “contador” ou
- Incorpora automaticamente o funcionamento de um “contador”
- Também pode percorrer uma lista através de um “iterador”
- O Python é responsável por controlar tudo automaticamente

Estruturas de Repetição

- Contada – Comando “for” com “range”

```
print("\nQuantidade fixa:")  
for x in range(5):  
    print(x)
```

```
print("\nInício e fim do intervalo:")  
for x in range(10, 15):  
    print(x)
```

```
print("\nInício, fim e passo do intervalo:")  
for x in range(21, 30, 3):  
    print(x)
```

```
print("\nInício, fim e passo do intervalo decrescente:")  
for x in range(30, 21, -3):  
    print(x)
```

- range(quantidade) - range(início, fim) - range(início, fim, passo)
- Somente valores inteiros
- Parâmetro “início” pertence à sequência, “fim” não

Estruturas de Repetição

- Contada – Comando “for” com “iterador”

```
print("\nColeção de números fixos")
for x in (1, 2, 3, 5, 8):
    print("Número:", x)

print("\nColeção de textos fixos")
for x in ['banana', 'laranja', 'abacaxi', 'uva']:
    print("Fruta:", x)
```

- A cada iteração, a variável de controle assume um por um dos valores da lista

Estruturas de Repetição

- **Condicional**
 - Realiza repetições em **quantidades indeterminadas**
 - Executa **enquanto** uma **condição** de repetição é atendida
 - Variável de controle do tipo “acumulador” ou “registrador”
 - Utiliza uma expressão lógica para decidir se permanece em execução
 - Avalia a expressão lógica a cada iteração do laço
 - O programador é responsável por controlar tudo manualmente

Estruturas de Repetição

```
print("\nQuantidade fixa:")
```

```
i = 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1
```

```
print("\nInício e fim do intervalo:")
```

```
i = 10
```

```
while i < 15:
```

```
    print(i)
```

```
    i += 1
```

```
print("\nInício, fim e passo do intervalo:")
```

```
i = 21
```

```
while i < 30:
```

```
    print(i)
```

```
    i += 3
```

```
print("\nInício, fim e passo do intervalo  
    decrescente:")
```

```
i = 30
```

```
while i > 21:
```

```
    print(i)
```

```
    i -= 3
```

Estruturas de Repetição

```
print('\nDigitar valores até atingir um somatório:')
soma = 0
while soma < 100:
    soma += int(input('Digite um número: '))
print('A soma dos valores digitados é', soma)

print('\nSolicitar um valor pertencente a um domínio:')
sexo = ''
while sexo not in ('m', 'M', 'f', 'F'):
    sexo = input('Informe um sexo (m/f): ')
print('Sexo informado:', sexo)
```


Estruturas Aninhadas

- É possível combinar estruturas de repetição em 2 ou mais níveis
- Também pode-se usar estruturas de seleção dentro de laços

```
print('\nLaços aninhados')
for letra in range(ord('A'), ord('D')):
    for numero in range(1, 4):
        print(chr(letra), numero)
    print()
```

```
print('\nDigite 5 letras ou números')
for x in range(1, 6):
    valor = input('{}: '.format(x))
    if valor.isnumeric():
        print('Numero')
    else:
        print('Letra')
```

Intervenções no Fluxo

- **continue:** suspende a execução corrente e pula para o próximo valor da sequência ou reavalia a condição de permanência

```
print('\nImprime valores de 1 a 10 exceto múltiplos de 3:')
```

```
# Iterador
```

```
for x in range(1, 11):
```

```
    if x % 3 == 0:
```

```
        continue
```

```
    print(x)
```

```
# Contador - tem bug aqui!
```

```
x = 1
```

```
while x < 11:
```

```
    if x % 3 == 0:
```

```
        continue
```

```
    print(x)
```

Intervenções no Fluxo

- **break:** encerra a execução das instruções e transfere o controle para fora do laço

```
print('\nMenor divisor de um número')
nr = int(input('Informe um número inteiro: '))

# Iterador
for x in range(2, nr + 1):
    if nr % x == 0:
        break
print(nr, 'é divisível por', x)

# Contador
x = 2
while x < nr + 1:
    if nr % x == 0:
        break
    x += 1
print(nr, 'é divisível por', x)
```

Referências

- https://www.w3schools.com/python/python_operators.asp
- https://www.w3schools.com/python/python_math.asp
- https://www.w3schools.com/python/module_math.asp

Obrigado.

